

ECE264-002 Spring 2020 Exam 2
8-9 PM, Thursday, Mar 12th, 2020

1. Structure and Union

Consider the following program and answer question 1.A - 1.E.

```
1.  #include <stdio.h>
2.  #include <stdlib.h>
3.  #pragma pack(1) // tell compiler not to pad any space
4.  // assume sizeof(int) = 4, sizeof(a pointer) = 8
5.
6.  const int NUM_REC = 5;
7.  typedef union {
8.      int value;
9.      struct {
10.         int length;
11.         int width;
12.     };
13. } Rectangle;
14.
15. int main(int argc, char ** argv) {
16.     Rectangle* arr = malloc(NUM_REC * sizeof(*arr));
17.     if(arr == NULL) {
18.         return EXIT_FAILURE;
19.     }
20.
21.     for(int ind = 0; ind < NUM_REC; ind++) {
22.         arr[ind].value = ind * ind + 15;
23.     }
24.     Rectangle* rptr = arr;
25.     printf("Answer 1.A: %d\n", arr -> value);
26.     printf("Answer 1.B: %d\n", arr -> length);
27.
28.     for(ind = 1; ind < NUM_REC; ind += 2) {
29.         arr[ind].length = 3 * ind + 2;
30.         arr[ind].width = ind + 1;
31.     }
32.     rptr += 3;
33.     printf("Answer 1.C: %d\n", rptr -> value);
34.     printf("Answer 1.D: %d\n", rptr -> width);
35.
36.     // 1.E: If the memory is not released, how many
37.     // bytes of memory are leaked?
38.     free(arr);
39.     return EXIT_SUCCESS;
40. }
```

Answer:

1.A: 15

1.B: 15

1.C: 11

1.D: 4

1.E: 40

2. Singly Linked List

Consider the following program to construct, print, and destroy a singly linked list.

```
1.  // list.h
2.  #ifndef LIST_H
3.  #define LIST_H
4.  typedef struct _Node {
5.      int value;
6.      // 2.A: fix the code to specify the data type of next
7.      2.A next;
8.  } Node;
9.  #endif
```

```
1.  // q2list.c
2.  #include <stdio.h>
3.  #include <stdlib.h>
4.  #include "list.h"
5.
6.  Node* create_node(int v) {
7.      Node* node = malloc(sizeof(*node));
8.      node -> value = v;
9.      node -> next = NULL;
10.     return node;
11. }
12.
13. // destroy_list: remove nodes from head
14. void destroy_list(Node* head) {
15.     while (head != NULL) {
16.         Node* node = head -> next;
17.         // 2.B: fix the following code
18.         free ( 2.B );
19.         head = node;
20.     }
21. }
22.
23. void print_list(Node* node) {
24.     while (node != 2.C ) {
25.         printf("%d ", node -> value);
26.         node = node -> next;
27.     }
28.     printf("\n");
29. }
```

```

30.
31. int main(int argc, char * argv[]) {
32.     if (argc < 2) {
33.         return EXIT_FAILURE;
34.     }
35.
36.     Node* head = create_node((int) strtol(argv[1], NULL, 10));
37.     Node* tail = head;
38.     Node* new_node;
39.     for (int i = 2; i < argc; i++) {
40.         new_node = create_node((int) strtol(argv[i], NULL, 10));
41.         tail -> next = new_node;
42.         // 2.D: fix the code
43.         tail = 2.D ;
44.     }
45.
46.     print_list(head);
47.     destroy_list(head);
48.     return EXIT_SUCCESS;
49. }

```

Question 2.E: Suppose we compile q2list.c with the following command

```
gcc q2list.c -o q2prog
```

Run the executable program with the command

```
./q2prog 1 5 7
```

Please write and program output in 2.E.

Answer:

2.A: struct _Node*

2.B: head

2.C: NULL

2.D: new_node or tail -> next

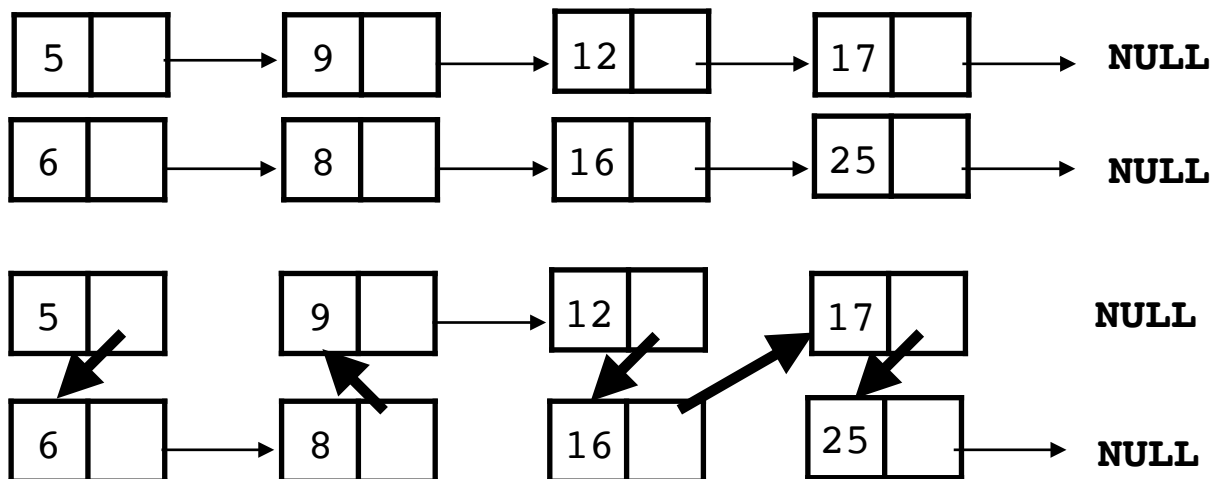
2.E: 1 5 7

3. Merge Two Sorted Linked Lists

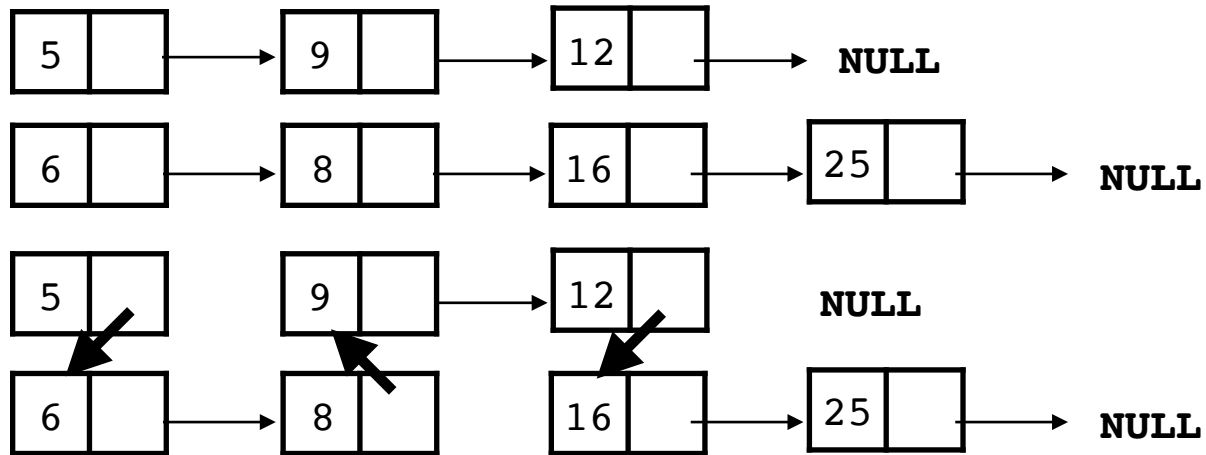
Consider the following recursive function to merge two sorted linked lists to one single sorted linked list. Please refer to `list.h` in Question 2 for the definition of `Node` in the linked list.

```
1.  Node* merge_sorted_list(Node* p, Node* q) {
2.      Node * head = NULL;
3.      if (p == NULL) {
4.          return q;
5.      }
6.      if (q == NULL) {
7.          return p;
8.      }
9.      if (p->value <= q->value) {
10.         head = p;
11.         head->next = merge_sorted_list(p->next, q);
12.     }
13.     else {
14.         head = q;
15.         head->next = merge_sorted_list(p, q->next);
16.     }
17.     return head;
18. }
```

The `merge_sorted_list` function merges two sorted linked lists by modifying some links. For example, merging the following two sorted linked lists needs to modify five links.



Merging the following two sorted linked list needs to modify three links.



Suppose the two sorted linked lists have distinct values (every value in the linked lists is unique). Denote the number of nodes in the first sorted linked list as m , and the number of nodes in the second sorted linked list as n .

- 3.A:** If $m = 3$, $n = 3$, what is the **minimum** number of modified links?
- 3.B:** If $m = 10$, $n = 10$, what is the **maximum** number of modified links?
- 3.C:** If $m = 9$, $n = 10$, what is the **maximum** number of modified links?
- 3.D:** If $m = 3$, $n = 100$, what is the **minimum** number of modified links?
- 3.E:** If $m = 3$, $n = 100$, what is the **maximum** number of modified links?

Answer:

3.A: 1

3.B: 19

3.C: 18

3.D: 1

3.E: 6

4. Recursive Program

Consider the following program in q4.c:

```
1.  #include <stdio.h>
2.  #include <stdlib.h>
3.
4.  void func (int n) {
5.      if (n > 0) {
6.          printf ("%d ", n);
7.          func (n-2);
8.          printf ("%d ", n-1);
9.      }
10. }
11.
12. int main(int argc, char * argv[]) {
13.     if (argc != 2) {
14.         return EXIT_FAILURE;
15.     }
16.     int n = (int) strtol(argv[1], NULL, 10);
17.     func(n);
18.     printf("\n");
19.     return EXIT_SUCCESS;
20. }
```

Suppose we compile q4recursion.c with the following command

```
gcc q4recursion.c -o q4prog
```

4.A: Execute the program in a terminal using `./q4prog 1`, what is the output?

4.B: Execute the program in a terminal using `./q4prog 2`, what is the output?

4.C: Execute the program in a terminal using `./q4prog 10`, what is the **last number** in the output?

4.D: Execute the program in a terminal using `./q4prog 15`, how many numbers are there in the output?

4.E: Execute the program in a terminal using `./q4prog 30`, how many numbers are there in the output?

Answer:

4.A: 1 0

4.B: 2 1

4.C: 9

4.D: 16

4.E: 30